TEAM 9 · 2026–2027

Adaptive Traffic Signal *Controller*

Using IoT, MQTT, Node-RED and Machine Learning Analytics

Amrita Vishwa Vidyapeetham, Amritapuri Campus

B.Tech CSE · Semester VII · IoT Mini-Project

Samanthula Eswar
AM.SC.U4CSE23349

Rayudu Vamsi Krishna
AM.SC.U4CSE23344

Bantupalli Harsha Vardhan
AM.SC.U4CSE23210

Kota Karthik
AM.SC.U4CSE23229

Sammeta Pavan
AM.SC.U4CSE23350

A software-based traffic simulation

Introduction & *objectives*

- Fixed-time signals repeat a preset cycle, whatever is queued.
- An IoT pipeline links sensing, messaging, storage, control and actuator feedback.

PROBLEM STATEMENT

Can an IoT-based controller read the traffic state, pick a bounded green time, apply it through an actuator and check the acknowledgement – and do better than fixed timing?

OBJECTIVES

1. IoT pipeline: simulation → messaging → storage → analytics → decision → actuation
2. Prepare a dataset: readings, signal cycles, adaptive decisions
3. Deterministic adaptive controller and a fixed-time baseline
4. Compare Logistic Regression, Random Forest, Gradient Boosting, MLP
5. Measure fit time, latency, memory and model size
6. Evaluate closed-loop queues, ACKs and signal safety
7. Dashboard and testing tools

Existing vs *proposed* system

ASPECT	FIXED-TIME BASELINE	PROPOSED SYSTEM
Green-time decision	Preset duration	Computed from current traffic within limits
Response to demand	No reaction during a run	Re-evaluates demand during operation
Feedback	No actuator feedback	Command acknowledgements recorded
Data handling	Fixed schedule	Readings, decisions, events in SQLite
Analytics	No classifier	Four offline ML classifiers evaluated
Testing	Schedule execution	Testing lab, fault checks, Stop/Reset validation

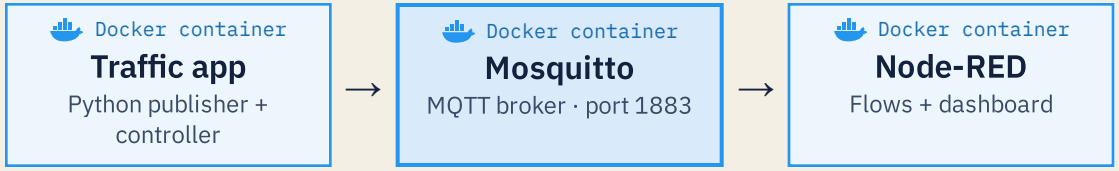
MQTT *communication*

HOW IT WORKS

- Publish–subscribe messaging protocol
- A publisher sends to a topic; the broker forwards it to subscribers
- Producers and consumers stay decoupled
- JSON messages with timestamps or IDs
- QoS levels trade delivery guarantees against overhead

COMPONENT	ROLE ON THE BROKER
Python simulator	Publishes traffic readings
Node-RED	Subscribes, validates, stores
Controller	Publishes signal commands
Actuator	Publishes ACKs and status

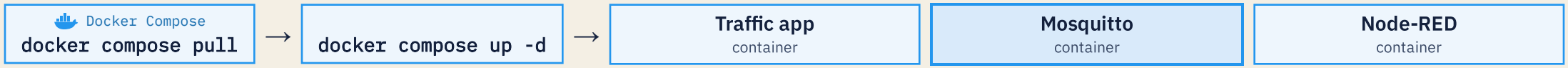
Readings | Commands | Acknowledgements | Status



Separate topic branches · image tag 1.0.0 · MQTT 1883 stays internal

Technologies *used*

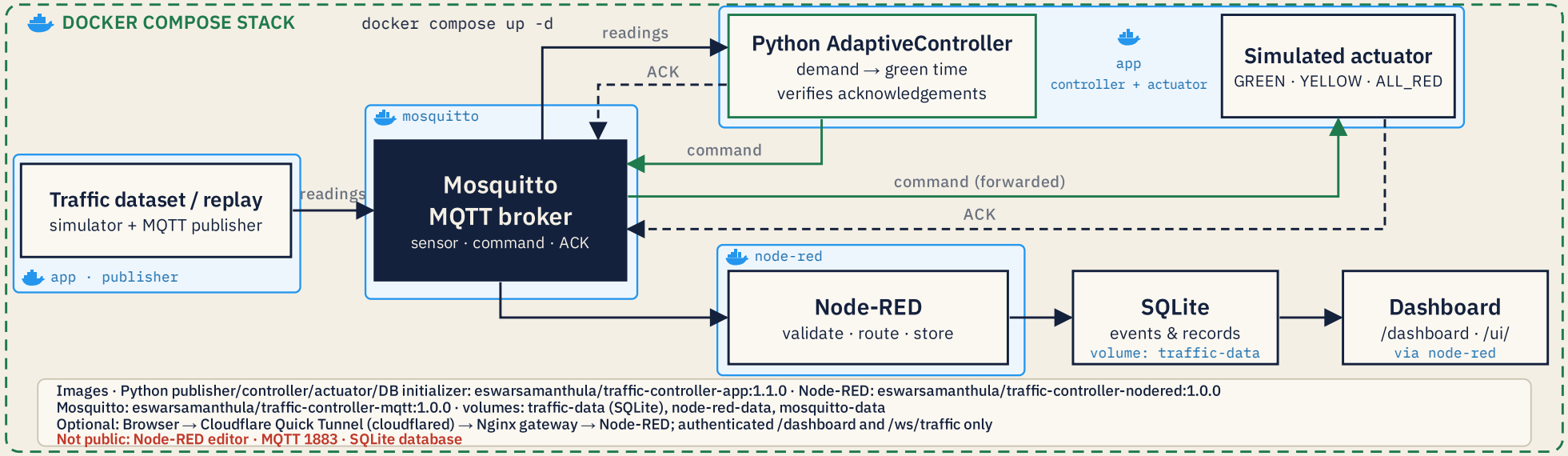
LAYER	TECHNOLOGY / TOOL	ROLE
Programming	Python 3.13.7	Simulation, controller, analytics, tests
Messaging	MQTT · Mosquitto 2.1.2	Publish–subscribe messaging and broker
Flow integration	Node-RED 5.0.1	Ingestion, validation, DB integration, dashboard
Runtime	Node.js 24.18.0	Node-RED runtime
Storage	SQLite	Readings, decisions, events, ACKs
Machine learning	scikit-learn 1.7.2	Offline model comparison
Numerical	NumPy 2.2.6	Data and numerical operations
Data files	Excel and CSV	Dataset workbook and script inputs
Deployment	Docker Compose	Shared Python app image; separate Mosquitto and Node-RED images



Images: eswarsamanthula/traffic-controller-app:1.1.0 · traffic-controller-nodered:1.0.0 · traffic-controller-mqtt:1.0.0

Windows 11 · 12 logical CPUs · CPU-only · no physical hardware claimed

Architecture & *workflow*



1 Simulator replays one reading per approach per minute
app

2 Mosquitto distributes readings
mosquitto

3 Node-RED validates and stores in SQLite
node-red

4 Dashboard shows traffic, signals, decisions
node-red

5 Controller calculates the next green duration
app

6 Command published through MQTT
app · mosquitto

7 Actuator applies GREEN → YELLOW → ALL_RED
app

8 ACK published and recorded
app · node-red

Node-RED, SQLite & *adaptive control*

NODE-RED

- Subscribes to MQTT messages
- Checks required fields and ranges
- Valid records → SQLite; invalid handled separately
- Feeds dashboard and testing controls

SQLITE

- Traffic readings
- Controller decisions
- Signal events
- Acknowledgement information

Persistent record for dashboard, tests and evaluation. Docker volume: traffic-data, shared with the Python app.

ADAPTIVE SIGNAL CONTROL

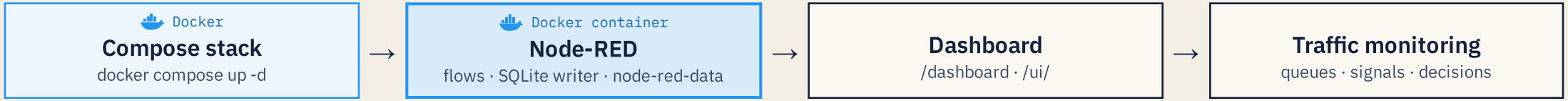
`Requested green = previous green × (approach demand ÷ average demand)`

Demand: queue length · arrival rate · waiting time

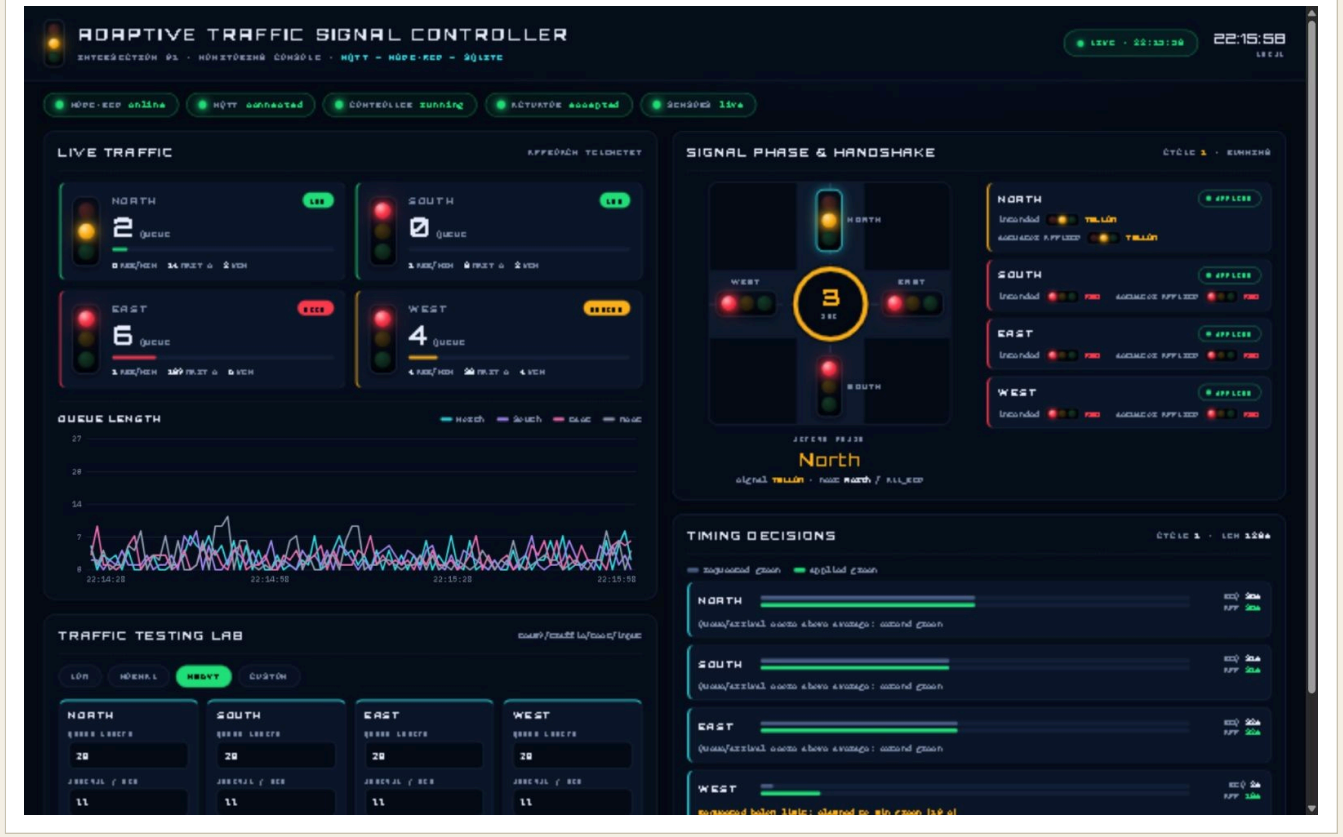
10 s	25 s	60 s	3 s	2 s
min green	default	max green	yellow	all-red

Actuator enforces GREEN → YELLOW → ALL_RED, rejects unsafe commands and returns an ACK.

DEPLOYMENT



Monitoring *dashboard*



- Four approach cards with queues and signal lamps
- Signal phase, countdown and handshake view
- Queue-length trend
- Requested vs applied green time, with reason
- System status; Testing Lab with Low, Normal, Heavy, Custom

Served by the node-red container · optional authenticated gateway routes `/dashboard` and `/ws/traffic` only; the editor stays private.

Dashboard running through Dockerized Node-RED deployment

Dataset & *ML algorithms*

5,000 traffic readings	***1,250*** one-minute timestamps × 4 approaches	***4,996*** feature rows after lags	***3,492 / 1,500*** train / test (4 boundary rows excluded)

PREPARATION & SPLIT

- Team-prepared dataset; 03:00–23:49, 12 Oct 2026
- Linked sheets: readings, signal cycles, decisions, fixed baseline
- Features: lags, rolling averages, change, time of day (past data only)
- Chronological split; test: Low 67.80% · Medium 25.67% · High 6.53%

FOUR OFFLINE CLASSIFIERS

- **Logistic Regression** — linear baseline
- **Random Forest** — ensemble of decision trees
- **Gradient Boosting** — sequential tree ensemble
- **MLP** — neural network, layers 64 and 32

Offline analytics only; live control is rule-based.

Logistic *Regression*

ABOUT THE MODEL

- Linear classification baseline
- Scores each congestion class from a weighted sum of the features
- Simple, fast and easy to interpret

CONFIGURATION

- StandardScaler on features
- max_iter = 1,000

TEMPORAL TEST SET (1,500 ROWS)

0.6987 Accuracy	**0.4238** Balanced accuracy
0.4384 Macro F1	**0.1327** High-class recall

COMPUTATIONAL COST

Fit time (mean of 3)	0.0253 s
Peak process RSS	153.9 MB
Latency p50 / p95	0.094 / 0.191 ms
Saved artifact	0.030 MB

Shortest fit time and smallest saved artifact of the four models.

Random *Forest*

ABOUT THE MODEL

- Ensemble of decision trees
- Trees vote; the forest combines the votes
- Captures non-linear feature interactions

CONFIGURATION

- 300 trees
- max_features = sqrt; unweighted classes

TEMPORAL TEST SET (1,500 ROWS)

0.6713 Accuracy	**0.4538** Balanced accuracy
0.4747 Macro F1	**0.2143** High-class recall

COMPUTATIONAL COST

Fit time (mean of 3)	0.7969 s
Peak process RSS	249.5 MB
Latency p50 / p95	4.912 / 6.806 ms
Saved artifact	53.972 MB

Largest saved artifact and highest p50/p95 inference latency of the four models.

Gradient Boosting: *algorithm*

IDEA

- Sequential ensemble of decision trees
- Each new tree is trained to reduce the errors left by the trees before it
- Tree outputs are added together, scaled by the learning rate
- Shallow trees keep each learner simple

SEQUENTIAL BUILDING

Tree 1 → Tree 2 → · · · → Tree 150

Each tree focuses on what the earlier trees got wrong.

Sum of trees → Low / Medium / High

150	*0.05*	*2*
estimators	learning rate	maximum tree depth

Gradient Boosting: *working & training*

1 Engineered features
Queue and flow, lags, rolling averages, change from previous minute, time of day — past data only

2 Temporal split
3,492 earlier rows train · 1,500 later rows test · 4 boundary rows excluded

3 Fit on training rows
150 trees added one after another; mean fit time 0.7152 s over 3 runs

4 Predict congestion class
Low, Medium or High for each test row

5 Evaluate
Accuracy, balanced accuracy, macro F1, High-class recall

Gradient Boosting: *results & performance*

0.7067	0.4163	0.4258	0.0714
Accuracy · highest of the four	Balanced accuracy · lowest	Macro F1 · lowest	High-class recall · lowest

COMPUTATIONAL BENCHMARK

0.7152 s	270.7 MB	0.342 ms	0.611 ms	0.361 MB
fit time	peak process RSS	latency p50	latency p95	saved artifact

Best overall accuracy (majority baseline 0.6780), but the weakest minority-class detection: High-class recall is only 7.14%.

MLP *neural network*

ABOUT THE MODEL

- Feed-forward neural network
- Two hidden layers of 64 and 32 units
- Learns non-linear combinations of the features

CONFIGURATION

- hidden layers (64, 32)
- alpha = 0.0001 · max_iter = 500

TEMPORAL TEST SET (1,500 ROWS)

0.6773 Accuracy	*0.4815* Balanced accuracy
0.5014 Macro F1	*0.2857* High-class recall

COMPUTATIONAL COST

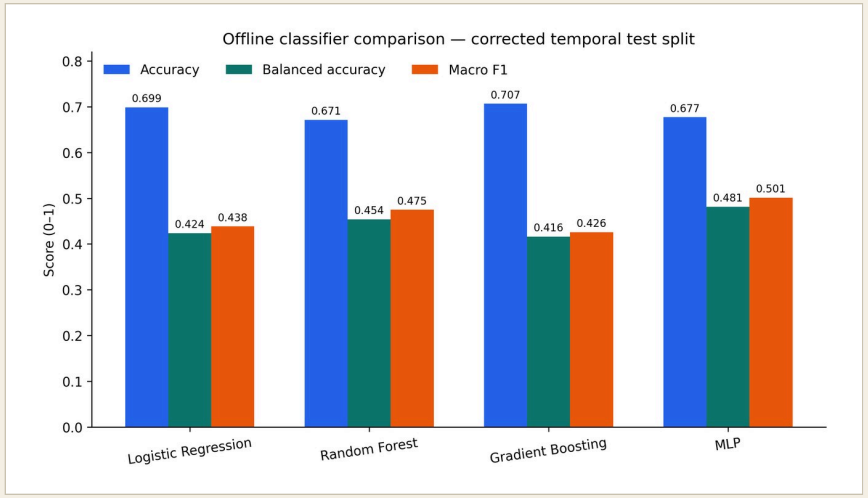
Fit time (mean of 3)	2.3105 s
Peak process RSS	273.5 MB
Latency p50 / p95	0.110 / 0.128 ms
Saved artifact	0.129 MB

Best macro F1 and High-class recall, but the slowest fit and highest memory; accuracy sits just below the 0.6780 baseline.

MLP results & *model comparison*

MODEL	ACC.	BAL. ACC.	MACRO F1	HIGH REC.
Majority baseline	0.6780	—	—	—
Logistic Reg.	0.6987	0.4238	0.4384	0.1327
Random Forest	0.6713	0.4538	0.4747	0.2143
Gradient Boosting	**0.7067**	0.4163	0.4258	0.0714
MLP	0.6773	**0.4815**	**0.5014**	**0.2857**

MODEL	FIT (S)	PEAK RSS (MB)	P50 / P95 (MS)	ARTIFACT (MB)
Logistic Reg.	0.0253	153.9	0.094 / 0.191	0.030
Random Forest	0.7969	249.5	4.912 / 6.806	53.972
Gradient Boosting	0.7152	270.7	0.342 / 0.611	0.361
MLP	2.3105	273.5	0.110 / 0.128	0.129



- Best model depends on the metric
- High-class recall is low for every model
- ML is offline; live control is rule-based

Testing, safety & *conclusion*

CLOSED-LOOP TESTING · FIXED-TIME VS ADAPTIVE

MEASURE	AVG DELAY	AVG QUEUE	MAX QUEUE	JAIN FAIRNESS	MAX RED WAIT
Fixed-time	139.4 s	7.005	113	0.957	95 s
Adaptive	76.6 s	3.848	21	0.992	**164 s**

Simulation: shorter delay, longer worst-case red wait.

SAFETY / FALLBACK

- **37/37** phase ACKs accepted; no violations on successful paths
- **4/4** RED ACKs confirm Stop/Reset
- One ACK withheld → unconfirmed, restart inhibited
- **200** automated tests passed, 0 failed

DOCKER DEPLOYMENT VERIFICATION

- ✓ Containers started successfully
- ✓ MQTT communication verified
- ✓ Node-RED dashboard accessible
- ✓ Persistent volumes configured
- ✓ WebSocket traffic verified

DOCKER SECURITY

- Node-RED editor not publicly exposed
- MQTT port not publicly exposed
- SQLite kept internal
- Authenticated dashboard access
- Only required services exposed

Conclusion. The system provides adaptive signal control in simulation using IoT, MQTT, Node-RED and SQLite, with offline ML analytics. Delay and queues fell; every safe-state command is verified by acknowledgement. Dockerization improved deployment consistency by packaging the application, MQTT broker, and Node-RED environment into reproducible containers.

References

[1] OASIS, “MQTT Version 5.0,” OASIS Standard, March 2019.

[2] R. A. Light, “Mosquitto: server and client implementation of the MQTT protocol,” J. Open Source Software, vol. 2, no. 13, p. 265, 2017.

[3] OpenJS Foundation, “Node-RED: Low-code programming for event-driven applications,” nodered.org

[4] SQLite Consortium, “SQLite documentation,” sqlite.org/docs.html

[5] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[6] F. V. Webster, “Traffic signal settings,” Road Research Technical Paper No. 39, HMSO, London, 1958.

[7] Project source: https://github.com/eswarsamanthula/Adaptive_Traffic_Signal_Controller_Docker.git

Thank *You*